Web-Based Phishing Detection System

Project Proposal

Supervisor
Sir Khalid Charran

Co-supervisor
-

Submitted by

Muhammad Ibtehaj Mubarak
{2278-2022 / IT-22-205 }

Muhammad Zaid Bin Saud
{1768-2022 / IT-22-167 }

Muhammad Azaan Rashid
{2996-2022 / IT-22-425 }

Department of Computer Science,
Hamdard University, Karachi.

May 23, 2025

1. Introduction

❖ Phishing Threats in Web Environments

Phishing attacks continue to be a growing cybersecurity concern, especially for web users who often interact with suspicious links via email, social media, or insecure websites. These attacks may lead to identity theft, financial loss, or unauthorized access to sensitive data. Unfortunately, many users lack the awareness or tools to recognize these threats in real time while browsing online.

❖ Rule-Based Detection Approach

To address this issue, we propose a web application that can scan, detect, and block phishing links. The system uses rule-based logic and blacklist APIs to analyze URLs and identify common phishing patterns.

Features like IP-based URLs, use of suspicious symbols (e.g., "@" or "-"), absence of HTTPS, or unusually long domains help mark links as dangerous. This method ensures the web app remains lightweight, fast, and does not rely on heavy background processing.

❖ System Architecture Overview

The system is developed using HTML, CSS, and JavaScript for the frontend, while Python (Flask) is used for the backend. Users can manually enter a URL in the web interface, which is sent to the backend for analysis.

Based on the result, a warning message is displayed. A rule-based chatbot is also integrated on the frontend to provide immediate feedback and security tips based on the scan results.

❖ Future Integration of Machine Learning (ML)

To enhance the application, we plan to integrate Machine Learning for smarter phishing detection. For this, we will use the **Random Forest Classifier**, trained on datasets from sources like Kaggle or PhishTank.

The model will evaluate features such as URL length, symbol usage, domain structure, and other relevant factors. Once trained, the model will be deployed through a backend API to deliver faster predictions to the frontend.

❖ AI-Powered Chatbot Enhancement

The existing rule-based chatbot can be further upgraded to an AI-driven conversational assistant using tools such as Dialogflow, Rasa, or the ChatGPT API. This would allow intelligent interactions, helping users better understand the nature of phishing threats and how to avoid them.

❖ Conclusion and Project Goal

In conclusion, our goal is to develop a secure, browser-accessible web application that enables users to detect phishing threats in real-time. With plans for ML and AI integration, the system aims to offer smarter and more interactive threat prevention.

2. Objective

- To Detect Phishing Links in Real-Time

The primary objective of this project is to develop a web-based phishing detection platform that can instantly analyze and identify phishing links. Most users come across malicious URLs while browsing websites, checking emails, or accessing unsecured web pages.

This application will inspect link structures using rule-based logic and blacklist APIs to determine if a URL is safe or potentially harmful.

- To Block Suspicious and Malicious Links

When a link is flagged as phishing, the web app will block or restrict user access to that site. Rather than allowing the user to proceed unknowingly, the system will display a warning message or redirect them to a safe informational page. This reduces the risk of accidentally interacting with harmful sites.

- To Provide User Alerts and Warnings

A key function of the web application is to inform users whenever a suspicious or harmful link is detected. Real-time alerts and warning messages will help users understand the risks, enhancing their awareness and enabling better decision-making when clicking on links.

- To Include a Built-In Chatbox for Assistance

The application features a simple chat box that provides direct feedback based on the results of the link scan. This rule-based chat assistant responds with predefined messages according to the scan status.

In the future, the chatbox can be upgraded to an intelligent AI-driven system using Dialogflow, Rasa, or ChatGPT API for more dynamic and user-friendly support.

- To Keep the Web App Lightweight and Efficient

A major goal is to ensure the application remains fast and efficient when accessed via browsers.

As it does not initially depend on heavy ML models, the system delivers quick results without overloading system resources, making it practical for all users regardless of device capabilities.

- To Enable Future Integration of Machine Learning

While the current model operates on rule-based detection, the backend is structured in a way that allows easy integration of a trained Machine Learning model in future versions.

This scalability makes the web application intelligent, adaptive, and ready for evolving cyber threats.

3. Problem Description

❖ Increasing Threat of Phishing Attacks

Phishing remains one of the most widespread and dangerous forms of cyberattacks across the internet. Attackers often design fake websites or deceptive links that appear legitimate and trick users into clicking them.

Once accessed, these links can steal sensitive information like login credentials, credit card data, or personal identity details. This issue is especially serious on web platforms, where users often click links from emails, pop-ups, or social media without fully verifying them.

❖ Lack of Awareness Among Web Users

Many web users are not fully aware of how phishing attacks operate. They may unknowingly click on suspicious links embedded in emails, online ads, or messaging platforms.

Most browsers or websites do not provide built-in systems to detect or block these threats in real-time. As a result, users remain vulnerable to scams, identity theft, and data breaches.

❖ Absence of Lightweight, Real-Time Detection Tools

Although some advanced phishing protection tools exist, they are often bundled with heavy enterprise-level antivirus software or browser extensions. These tools may require complex setup, additional costs, or high system resources.

There is a noticeable lack of lightweight, browser-accessible platforms that offer instant phishing link detection without overwhelming the user or slowing down the system.

❖ Need for an Interactive, Supportive System

Even when a link is identified as malicious, users are often confused about what steps to take next. A major gap exists in terms of user interaction and support after detection.

A system that can not only detect phishing threats but also communicate with users in a helpful and informative way can greatly enhance online safety.

❖ Solution Direction

Our project addresses these concerns by developing a web application that analyzes links using rule-based logic and blacklist APIs. The system blocks malicious URLs and provides instant support through a built-in chatbot.

It is designed to be simple, user-friendly, and prepared for future integration of Machine Learning or AI for smarter detection and interaction.

4. Comprehensive Literature Review

Phishing detection continues to be an active area of cybersecurity research, with a range of techniques proposed, such as rule-based logic, heuristic models, blacklist systems, and machine learning. Since our web app relies on rule-based detection and blacklist APIs, this review explores relevant studies supporting our approach.

❖ Rule-Based Detection Approaches

Basnet et al. (2015) proposed a phishing detection method that uses rule-based logic to analyze website behavior and identify phishing attempts. The system examined attacker tactics and developed rules to distinguish phishing from legitimate webpages. Their approach achieved an impressive accuracy of 95–99% with a false positive rate as low as 0.5–1.5%.

This high accuracy proves the effectiveness of rule-based systems, especially when rules are carefully designed. It also validates our choice of using a rule-based engine in our mobile app to detect malicious links.

❖ Heuristic and Blacklist Hybrid Schemes

Researchers at Temple University introduced "MobiFish," a mobile anti-phishing system that relies on heuristic rules combined with blacklist checks. Their research showed that while these approaches are effective in many cases, they still struggle with detecting zero-day phishing attacks (previously unknown threats).

However, for known and pattern-based phishing links, the combination of heuristics and blacklist APIs is still one of the most reliable and lightweight solutions, making it ideal for mobile apps like ours.

❖ Rule-Based Web Detection Systems

Recent implementations (referenced via Gale) designed for browser-based detection rely on lightweight rules to analyze URL structures and flag suspicious links in real-time. These systems are easy to integrate into web applications and support low-latency responses, reinforcing the practicality of our approach

❖ Comprehensive Survey of Detection Techniques

Dholakia and Agrawal (2018) reviewed many phishing detection methods and concluded that no single method solves all threats.

Blacklist and rule-based systems effectively handle known attacks, but zero-day and sophisticated phishing require machine learning or deep learning integration. This supports our plan to begin with rule-based logic and later integrate ML for enhanced detection.

❖ Rule-Based SMS-Based Detection (Analogous Techniques)

Akande et al. (2021) developed a real-time smishing detector using rule-based classifiers like RIPPER and C4.5. Although SMS-focused, the underlying method, scanning textual links for suspicious patterns, is directly applicable to web URLs, especially in resource-constrained environments.

5. Methodology

The methodology of this project is based on the **Waterfall Model**, a traditional and linear software development approach where each phase must be completed before moving to the next.

This model is ideal for academic-level web development projects where the requirements are clearly defined from the start. Since our phishing detection web app has specific objectives and features, the Waterfall Model ensures structured planning, implementation, and delivery.

❖ Phase 1: Requirement Gathering

In the first phase, we'll collect both functional and non-functional requirements for the system. This included understanding how phishing links behave, which rule-based patterns to detect, how users would interact with the web interface, and how the chatbot would assist them.

We will also finalize the technologies to be used like, HTML/CSS and JavaScript for the frontend, and Python (Flask) for the backend processing.

Additionally, we will explore the feasibility of integrating a **Machine Learning** model for more intelligent phishing detection based on real-world URL datasets (e.g., from Kaggle or PhishTank).

❖ Phase 2: System Design

This stage focused on designing the system's architecture. It involved planning the user interface for link input, backend APIs for analyzing links using rule-based logic, and integration with blacklist APIs (e.g., Google Safe Browsing or PhishTank).

A chatbot module will also design for user interaction. In parallel, we will identify and define the features to be extracted for ML, such as URL length, use of special characters, presence of IP addresses, and keyword patterns.

A plan will be made to connect the ML model **(Random Forest Classifier)** to the backend process.

❖ Phase 3: Implementation

We will built the frontend using HTML, CSS, and JavaScript to create a user-friendly web interface. The backend will develope using Python and Flask, where phishing detection logic will be implemented using predefined rules and blacklist checks.

Apart from that, the ML model predicts whether a link is phishing or not based on extracted features. After analysis, results will be returned to the frontend, along with chatbot responses.

❖ Phase 4: Testing

We will test the system using different kinds of URLs — safe, shortened, phishing, and obfuscated. However, we will also evaluate the accuracy of both the rule-based logic and the ML predictions, ensuring that response handling and the user interface worked as expected.

❖ Phase 5: Deployment & Maintenance

After successful testing, the web app will deploy on a local server and made accessible via a browser. In the future, we plan to continue updating detection rules, retraining the ML model with updated datasets, and upgrading the chatbot to an AI-powered system (e.g., Dialogflow or ChatGPT API).

6. Project Scope

The scope of this project is to develop a lightweight, real-time web application that detects, blocks, and reports phishing links using rule-based logic.

The system is accessible through any modern browser and focuses on enhancing online safety by offering smart link scanning and interactive user support through a built-in chatbot.

❖ Core Functionality

The main functionality of the web application is to analyze and identify phishing URLs using predefined rule-based detection methods and integration with blacklist APIs.

Users can enter or paste any URL into the input field of the web interface, which is then sent to the backend for verification. If the link is determined to be phishing, it is blocked, and the user receives an immediate warning message.

❖ Target Users

The application is designed for general internet users who may not have deep technical knowledge about cybersecurity but are often exposed to phishing attempts while browsing, checking emails, or interacting on social platforms.

The tool is meant to be simple, browser-friendly, and easy to use, especially for users who need basic protection from malicious web links.

❖ System Components

The scope of the project includes the development of the following core components:

1. Frontend UI using HTML, CSS, and JavaScript for input and result display
2. Python Flask Backend to apply rule-based logic and process link scans
3. Integration with Blacklist APIs such as PhishTank or Google Safe Browsing
4. Rule-Based Chatbot to assist users with warnings and basic phishing education

❖ Limitations

This version of the project includes Machine Learning-based phishing detection using a trained Random Forest Classifier in the backend. While the model provides accurate predictions based on static URL features (like length, presence of symbols, or use of IP addresses), it does not yet handle dynamic webpage content, real-time behavior, or image-based phishing indicators.

Additionally, the chatbot is currently rule-based and offers predefined responses rather than understanding user queries through natural language processing. Future upgrades can focus on expanding the ML model's scope and enhancing the chatbot with AI for smarter user interaction.

❖ Future Enhancements

Future upgrades could include browser extension support for auto-detecting links, integration with AI chat platforms for smarter responses, and use of machine learning models to improve phishing prediction accuracy.

7. Feasibility Study

The goal of this feasibility study is to assess whether the development of the Web App Phishing Detection System is practical in terms of time, technical skills, and available resources. After analyzing all aspects, we conclude that the project is feasible and achievable within the given academic timeline using current web technologies and machine learning tools.

➢ Risks Involved:

1. False Positives/Negatives: Even with ML, the system might incorrectly classify safe links as phishing or miss some harmful ones, which could impact user trust.

2. API Limitations: Public blacklist APIs like PhishTank or Google Safe Browsing may enforce data access limits or rate restrictions during high usage.

3. Browser Compatibility: Since the system is web-based, it must function smoothly across all modern browsers. Differences in rendering or JavaScript support may impact user experience.

4. Model Performance: The Random Forest Classifier is trained on static features and may not detect advanced or zero-day phishing attacks involving obfuscated or dynamic content.

5. Security Threats: Proper backend validation and secure handling of user-submitted URLs must be ensured to avoid misuse or exposure to malicious input.

➢ Resource Requirement:

1. Human Resources:

    - 3 Developers (HTML/CSS/JS frontend, Python backend with ML & API integration)
    - 1 Supervisor (Technical and project guidance)

2. Software Tools:

    - HTML, CSS, JavaScript (Frontend)
    - Python (Flask framework)
    - Visual Studio Code
    - Browser DevTools (for testing and debugging)
    - GitHub (for version control)

3. Hardware:

    - Laptops with at least 8GB of RAM and a stable internet connection
    - Backup storage (external HDD or cloud)

4. APIs/Services:

    - PhishTank API
    - Google Safe Browsing API
    - Trained Random Forest Model (.pkl file served via Flask backend)

8. Solution Application Areas

The Phishing Detection System - Web App addresses a growing cybersecurity need in today's internet-driven world. Its application scope is wide and impactful, especially for users who frequently browse the web, access online platforms, or engage in email and social media interactions.

Below are the key areas where this web-based solution can be effectively applied:

- ❖ Web Security for General Internet Users

This application can be used by everyday users to scan and verify suspicious links received through emails, messaging platforms (e.g., WhatsApp Web, Messenger, Telegram), websites, or social networks.

It helps reduce the chances of falling prey to scams, identity theft, or fake login pages.

- ❖ Corporate and Employee Protection

Companies can adopt this web application as a lightweight internal tool or share it with employees, particularly remote workers, to ensure they verify links before accessing work-related resources. It strengthens protection against phishing-based attacks targeting organizational systems and confidential data.

- ❖ Cybersecurity Education for Students and Institutions

Educational institutions can include this web application in training programs to promote phishing awareness. Because of its simplicity and web-based access, it fits well into academic environments and digital safety campaigns without the need for installations.

- ❖ Financial and Online Transaction Users

Users frequently accessing online banking, payment gateways, or e-commerce platforms are common phishing targets. This tool allows them to pre-check URLs before logging into sensitive accounts, adding an extra security layer during online transactions.

- ❖ Support for Non-Technical or Elderly Users

Non-tech-savvy individuals, especially elderly users, often lack awareness about phishing risks. This easy-to-use web interface provides them with a reliable method to verify any link before proceeding, without requiring installation or technical knowledge.

- ❖ Future Integration in Browser Extensions or Antivirus Tools

With further development, this phishing detection tool could be integrated into web browser extensions or full-fledged security platforms. This would allow real-time scanning of clicked links, improving overall browsing safety across various environments.

9. Tools and Technology

❖ Software Tools:

1. Visual Studio Code – Code editor for frontend (HTML/CSS/JS) and backend (Python) development.
2. Flask / FastAPI – Python web frameworks used for developing and serving backend APIs.
3. MySQL – Relational database for optional storage/logging of scanned URLs or user interaction data.
4. TensorFlow – A Machine Learning library used to train and manage the phishing detection model.
5. GitHub – Version control and collaborative platform for source code management.

❖ Hardware Requirements:

1. Laptop – Multi-core Intel Core i9 processor, 16 GB RAM, 500 GB SSD for smooth local development and testing.
2. Backup Hard Drives – For safe storage and version backup of project files and datasets.
3. Internet Dongle – For reliable internet access during development and deployment.
4. Printer – For documentation printing, hardcopy reports, and presentations, if required.

10. Responsibilities of the Team Members

Modules	Supervisor	Ibtehaj	Zaid	Azaan
Proposal & Project Planning	C, I	A, R	R	A
Literature Review	C, I	R	A	A, R
UI Design (Web Frontend)	C	A	R	R
Rule-Based Detection Logic	C, I	R	A, R	A
Blacklist & Heuristic Integration	I	A, R	A	R
Python API - Frontend Comm.	C, I	A, R	R	A
Web App Testing & Debugging	C, I	A	A, R	R
Final Report & Presentation	C, I	A, R	R	A
Communication & Documents	C, I	A	R	R

11. Planning

Task ID	Task Name	Start Date	End Date	Duration
T1	Proposal & Project Planning	Aug 1, 2025	Aug 14, 2025	2 weeks
T2	Literature Review	Aug 18, 2025	Aug 25, 2025	1 week
T3	UI Design (Web Frontend)	Sep 1, 2025	Sep 30, 2025	1 month
T4	Rule-Based Detection Logic	Oct 5, 2025	Dec 5, 2025	3 months
T5	Blacklist & Heuristic Integration	Dec 15, 2025	Feb 20, 2026	2 months
T6	Python API - Frontend Comm.	Mar 1, 2026	Apr 1, 2026	1 month
T7	Web App Testing & Debugging	May 5, 2026	May 12, 2026	1 week
T8	Final Report & Presentation	May 15, 2026	Jun 1, 2026	15 days
T9	Communication & Documents	Jun 5, 2026	Jun 25, 2026	10 days

12. References & Research

1. Basnet, R. B., Sung, A. H., & Liu, Q. (2015). Rule-Based Phishing Attack Detection

https://www.researchgate.net/publication/265919217_Rule-Based_Phishing_Attack_Detection

2. MobiFish: A Lightweight Anti-Phishing Scheme for Mobile Phones

https://cis.temple.edu/~wu/research/publications/Publication_files/ICCCN14wlf.pdf

3. A Hybrid Super Learner Ensemble for Phishing Detection (referenced via Gale)

https://go.gale.com/ps/i.do?id=GALE%7CA499812753&issn=13529404&it=r&linkaccess=abs&p=AONE&sid=googleScholar&sw=w&v=2.1

https://www.nature.com/articles/s41598-025-02009-8

4. Dholakia, N., & Agrawal (2018), Review on Phishing Attack Detection Techniques

https://www.researchgate.net/publication/310492979_A_survey_and_classification_of_web_phishing_detection_schemes

5. Akande, O. A., et al. (2021). Rule-Based Smishing Detection Mobile Application

https://www.researchgate.net/publication/361680637_DEVELOPMENT_OF_A_RULE-BASED_PHISHING_WEBSITE_CLASSIFICATION_SYST